

```

python

# =====

# RASPBERRY PI 4 HEALTH MONITOR - PHILIPPINE TIME WITH FINGER TRIGGER

# =====

# FEATURES:

# - FINGER TRIGGER: Place finger on MAX30102 to start measurement

# - RESTART Button (GPIO 27, Pin 13) - Reset sensors/display ONLY

# - POWER Switch (GPIO 17, Pin 11) - Toggle ON/OFF position for shutdown

# - Patient Number ONLY resets at midnight (00:00) Philippine Time

# - Real-time accurate Philippine Time (UTC+8) timestamp

# - Buzzer sounds on ALL valid temperature readings (LOW, NORMAL, FEVER, HIGH)

# AND valid HR (60-130) AND SpO2 (>=95)

# - Temperature: Raw object reading (no ambient compensation)

# - LCD displays "Valid Reading" for all temperature statuses (LOW, NORMAL, FEVER, HIGH)

#

# PINOUT:

# - BUZZER: GPIO 18 (Pin 12) - PWM output

# - RESTART BUTTON: GPIO 27 (Pin 13) - Active LOW (press to GND)

# - POWER Switch: GPIO 17 (Pin 11) - Active LOW (switch ON = GND)

# - I2C SDA: GPIO 2 (Pin 3) - LCD & Temperature sensor (MLX90614)

# - I2C SCL: GPIO 3 (Pin 5) - LCD & Temperature sensor (MLX90614)

# - UART TX: GPIO 14 (Pin 🧐 - Printer

# - UART RX: GPIO 15 (Pin 10) - Printer

import time

```

```
import board
import busio
import serial
import os
import sys
import glob
import subprocess

from datetime import datetime, date
from zoneinfo import ZoneInfo


from RPLCD.i2c import CharLCD
from smbus2 import SMBus
from mlx90614 import MLX90614
import RPi.GPIO as GPIO


# Supabase integration
import requests
import json


# MAX30102 library
try:
    from max30102 import MAX30102
    import hrcalc
except ImportError:
    print("Warning: MAX30102 library not found")
    MAX30102 = None
```

```
hrcalc = None
```

```
# =====
```

```
# PHILIPPINE TIME CONFIGURATION
```

```
# =====
```

```
PH_TIMEZONE = ZoneInfo("Asia/Manila")
```

```
def get_ph_timestamp():
```

```
    """Get current timestamp in Philippine Time (UTC+8)"""
```

```
    return datetime.now(PH_TIMEZONE)
```

```
def format_ph_time(dt=None):
```

```
    """Format datetime for display/printing in Philippine Time"""
```

```
    if dt is None:
```

```
        dt = get_ph_timestamp()
```

```
    return dt.strftime("%Y-%m-%d %H:%M:%S")
```

```
def get_ph_time_short():
```

```
    """Get current time as HH:MM string (Philippine Time)"""
```

```
    return get_ph_timestamp().strftime("%H:%M")
```

```
def get_ph_date():
```

```
    """Get current date (Philippine Time)"""
```

```
    return get_ph_timestamp().date()
```

```
def is_midnight_ph():
```

```

""""Check if current time is midnight (00:00) in Philippine Time""""

now = get_ph_timestamp()

return now.hour == 0 and now.minute == 0


# =====

# FINGER TRIGGER SETTINGS

# =====

FINGER_TRIGGER_THRESHOLD = 30000    # IR threshold to trigger system startup
FINGER_TRIGGER_COUNT = 4           # Consecutive readings to confirm finger
FINGER_TRIGGER_CHECK_INTERVAL = 0.3 # Check every 300ms in standby


# =====

# VALID READING THRESHOLDS (for buzzer feedback)

# =====

VALID_TEMP_MIN = 36.0
VALID_TEMP_MAX = 37.4
VALID_HR_MIN = 60
VALID_HR_MAX = 180 # Changed from 100 to 180 to accommodate active people
VALID_SPO2_MIN = 95


# Temperature statuses that are considered "valid" for buzzer
VALID_TEMP_STATUSES = ["LOW", "NORMAL", "FEVER", "HIGH"] # All fever levels are valid


# =====

# SUPABASE CONFIGURATION

# =====

```

```
SUPABASE_URL = "https://hgtDKtoonaqiwmnenikb.supabase.co"
```

```
SUPABASE_ANON_KEY =
```

```
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZil6ImhndGRrdG9vbWV4aXdtbmVuaWtliiwicm9sZSI6ImFub24iLCJpYXQiOiJlE3NzQ1MTM4ODgsImV4cCI6ImJA5MDA4OTg4OH0.6NXKSNNy2sgf0tXwmYqv_PPK27KoWB3lCw93YPZx1do"
```

```
def send_vitals_to_supabase(patient_no, temperature, temp_status, heart_rate=None, spo2=None):
```

```
    """Send vital signs to Supabase after successful print"""
```

```
    url = f"{SUPABASE_URL}/rest/v1/raw_vitals"
```

```
    headers = {
```

```
        "apikey": SUPABASE_ANON_KEY,
```

```
        "Authorization": f"Bearer {SUPABASE_ANON_KEY}",
```

```
        "Content-Type": "application/json",
```

```
        "Prefer": "return=minimal"
```

```
    }
```

```
    current_time = get_ph_timestamp()
```

```
    data = {
```

```
        "patient_number": int(patient_no),
```

```
        "temperature": float(temperature) if temperature else None,
```

```
        "temp_status": str(temp_status) if temp_status else None,
```

```
        "heart_rate": int(heart_rate) if heart_rate else None,
```

```
        "spo2": int(spo2) if spo2 else None,
```

```
        "printed": True,
```

```
        "measured_at": current_time.isoformat()
```

```
    }
```

```
    try:
```

```

response = requests.post(url, headers=headers, json=data, timeout=10)

print(f" → Response: {response.status_code}")

if response.text:

    print(f" → Details: {response.text[:100]}")

if response.status_code == 201:

    print(f"✓ Vitals sent to cloud for Patient #{patient_no} at
{format_ph_time(current_time)}")

    return True

elif response.status_code == 401:

    print(f"✗ AUTH FAILED: Check your SUPABASE_ANON_KEY")

    print(f" Key starts with: {SUPABASE_ANON_KEY[:20]}...")

    return False

else:

    print(f"✗ Failed: HTTP {response.status_code}")

    return False

except Exception as e:

    print(f"✗ Network error: {e}")

    return False

```

```

# =====

```

```

# GPIO PIN DEFINITIONS

```

```

# =====

```

```

BUZZER_PIN = 18

```

```

RESTART_BUTTON_PIN = 27

```

```

POWER_SWITCH_PIN = 17

```

```
# =====
```

```
# GLOBAL FLAGS
```

```
# =====
```

```
last_print_time = 0
```

```
PRINT_COOLDOWN = 10
```

```
last_printed_temp = None
```

```
last_printed_hr = None
```

```
last_printed_spo2 = None
```

```
current_patient_no = 1
```

```
last_date_check = None
```

```
daily_reset_done = False
```

```
restart_requested = False
```

```
shutdown_requested = False
```

```
startup_complete = False
```

```
power_switch_enabled = False
```

```
restart_button_pressed = False
```

```
# =====
```

```
# TEMPERATURE DETECTION SETTINGS
```

```
# =====
```

```
BODY_DETECTION_THRESHOLD = 28.0
```

```
BODY_DETECTION_SAMPLES = 5
```

```
BODY_DETECTION_TIMEOUT = 30
```

```
# =====
```

```
# POWER SWITCH FUNCTIONS
```

```

# =====

def safe_shutdown():
    """Perform safe shutdown of Raspberry Pi"""
    current_time = format_ph_time()
    print(f"\n  SAFE SHUTDOWN INITIATED at {current_time} (PH Time)")
    print(" Saving data and shutting down system...")
    if lcd:
        display_message(lcd, "SHUTTING DOWN", "Please wait...", 2)
        lcd.backlight_enabled = False
    beep(0.3, 3, 800)
    cleanup_gpio()
    if buzzer_pwm:
        buzzer_pwm.stop()
    if printer and printer.is_open:
        try:
            printer.write(b'\n\n\n\n\n\n')
            printer.flush()
            time.sleep(0.5)
            printer.close()
        except:
            pass
    print(" Executing shutdown command...")
    subprocess.call(['sudo', 'shutdown', '-h', 'now', 'System halted by power switch'])

def check_power_switch():
    """Check if power switch is in OFF position (LOW) - REQUIRES 5 second hold"""

```



```

global shutdown_requested

if not startup_complete or not power_switch_enabled:

    return False

if GPIO.input(POWER_SWITCH_PIN) == GPIO.LOW:

    press_start = time.time()

    while GPIO.input(POWER_SWITCH_PIN) == GPIO.LOW:

        held_time = time.time() - press_start

        if lcd:

            lcd.clear()

            lcd.cursor_pos = (0, 0)

            lcd.write_string("Switch OFF")

            lcd.cursor_pos = (1, 0)

            remaining = max(0, 5 - int(held_time))

            lcd.write_string(f"Shutdown in {remaining}s ")

        if held_time >= 5:

            shutdown_requested = True

            return True

        time.sleep(0.1)

    if lcd:

        display_message(lcd, "Switch ON", "Cancelled", 1)

    return False

return False

```

```

# =====

# DAILY PATIENT NUMBER RESET

# =====

```

```

def check_daily_reset():
    global current_patient_no, last_date_check, daily_reset_done
    today = get_ph_date()
    current_time = get_ph_timestamp()
    if last_date_check is None:
        last_date_check = today
        daily_reset_done = False
        return
    if today != last_date_check:
        if current_time.hour == 0 and current_time.minute < 5:
            if not daily_reset_done:
                current_patient_no = 1
                last_date_check = today
                daily_reset_done = True
                current_time_str = format_ph_time()
                print(f"\n 🌅 DAILY RESET at {current_time_str} (PH Time)")
                print(f" Patient counter reset to #{current_patient_no}")
                beep(0.2, 3, 1500)
                if lcd:
                    display_message(lcd, "Daily Reset!", f"Patient #1 Ready", 3)
            else:
                last_date_check = today
                daily_reset_done = False
        else:
            if not (current_time.hour == 0 and current_time.minute < 5):
                daily_reset_done = False

```

```

def force_restart_sensors():
    current_time = format_ph_time()

    print(f"\n 🔄 RESTART BUTTON PRESSED at {current_time} (PH Time)")
    print(f" Sensors reset - continuing from Patient #{current_patient_no}")

    if lcd:
        display_message(lcd, "Sensors Reset", f"Patient #{current_patient_no}", 2)

    beep(0.1, 2, 2000)

```

```

# =====

```

```

# RESTART BUTTON FUNCTIONS

```

```

# =====

```

```

def check_restart_button():
    """Poll restart button - NO INTERRUPT to avoid edge detection errors"""

    global restart_requested, restart_button_pressed

    if not startup_complete:
        return False

    if GPIO.input(RESTART_BUTTON_PIN) == GPIO.LOW:
        if not restart_button_pressed:
            restart_button_pressed = True

            restart_requested = True

            current_time = format_ph_time()

            print(f"\n 🔄 RESTART BUTTON PRESSED at {current_time}")

            beep(0.1, 3, 2500)

            return True

    else:

```

```

        restart_button_pressed = False

    return False

# =====

# SETUP GPIO

# =====

def setup_gpio():
    """Setup GPIO pins - NO INTERRUPTS to avoid edge detection errors"""
    try:
        GPIO.setup(POWER_SWITCH_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
        time.sleep(0.1)
        GPIO.setup(RESTART_BUTTON_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
        time.sleep(0.1)
        print(f"✓ GPIO pins configured (POLLED - no interrupts):")
        print(f" - POWER Switch: GPIO {POWER_SWITCH_PIN} (Pin 11)")
        print(f" - RESTART Button: GPIO {RESTART_BUTTON_PIN} (Pin 13)")
        power_state = GPIO.input(POWER_SWITCH_PIN)
        restart_state = GPIO.input(RESTART_BUTTON_PIN)
        print(f" - Initial states: POWER={power_state}, RESTART={restart_state}")
        return True
    except Exception as e:
        print(f"X GPIO setup error: {e}")
        return False

def cleanup_gpio():
    """Cleanup GPIO - no interrupts to remove"""

```

```
pass
```

```
# =====
```

```
# PATIENT NUMBER MANAGEMENT
```

```
# =====
```

```
def get_current_patient_no():
```

```
    global current_patient_no
```

```
    check_daily_reset()
```

```
    return current_patient_no
```

```
def increment_patient_no():
```

```
    global current_patient_no
```

```
    current_patient_no += 1
```

```
    current_time = format_ph_time()
```

```
    print(f"Patient number incremented to #{current_patient_no} at {current_time}")
```

```
    return current_patient_no
```

```
def display_patient_menu(lcd):
```

```
    if not lcd:
```

```
        return
```

```
    lcd.clear()
```

```
    lcd.cursor_pos = (0, 0)
```

```
    lcd.write_string(f"Patient #{current_patient_no:<4d}")
```

```
    lcd.cursor_pos = (1, 0)
```

```
    lcd.write_string(f"{get_ph_time_short()} PH Ready")
```

```

# =====

# PRINTER SETUP

# =====

def find_printer_port():
    ports = []
    possible_ports = [
        '/dev/serial0', '/dev/ttyAMA0', '/dev/ttyS0',
        '/dev/ttyUSB0', '/dev/ttyUSB1',
    ]
    for port in possible_ports:
        if os.path.exists(port):
            ports.append(port)
    return ports

def test_printer_connection(port, baudrate=9600, timeout=2):
    try:
        test_serial = serial.Serial(
            port=port, baudrate=baudrate, timeout=timeout, write_timeout=2
        )
        time.sleep(0.5)
        test_serial.reset_input_buffer()
        test_serial.reset_output_buffer()
        test_serial.write(b'\x1b@')
        test_serial.flush()
        time.sleep(0.1)
        test_serial.close()

```

```

        return True

except:

    return False


def initialize_printer():

    print("\n" + "="*50)

    print("THERMAL PRINTER INITIALIZATION")

    print("="*50)

    available_ports = find_printer_port()

    if not available_ports:

        print("X No serial ports found!")

        return None

    print(f"Found ports: {available_ports}")

    working_port = None

    for port in available_ports:

        print(f"\nTesting {port}...")

        if test_printer_connection(port):

            print(f" ✓ Printer detected on {port}")

            working_port = port

            break

    else:

        print(f" X No response from {port}")

    if not working_port:

        print("\nX PRINTER NOT DETECTED!")

        return None

try:

```

```

printer = serial.Serial(
    port=working_port, baudrate=9600,
    bytesize=serial.EIGHTBITS, parity=serial.PARITY_NONE,
    stopbits=serial.STOPBITS_ONE, timeout=1, write_timeout=5
)
time.sleep(0.5)
printer.write(b'\x1b@')
printer.write(b'\x1b2')
printer.write(b'\x1b!\x00')
printer.flush()
time.sleep(0.2)
print(f"\n✓ Thermal printer initialized on {working_port}")
return printer
except Exception as e:
    print(f"X Failed to initialize printer: {e}")
    return None

# =====
# PRINTING FUNCTION
# =====

def print_with_progress(printer, temp, status, hr, spo2, lcd=None, patient_no=None):
    global last_print_time, last_printed_temp, last_printed_hr, last_printed_spo2
    if not printer or not printer.is_open:
        print("X Printer not available")
        if lcd:
            display_message(lcd, "Print Failed", "No printer", 2)

```



```
    return False

if temp is None:

    print("X Cannot print: Missing temperature data")

    if lcd:

        display_message(lcd, "Print Blocked", "No temp data", 2)

    return False

if temp < 30 or temp > 45:

    print(f"X Cannot print: Invalid temp ({temp:.1f}°C)")

    if lcd:

        display_message(lcd, "Invalid Temp", "Cannot print", 2)

    return False

if sensor is not None:

    if hr is None or spo2 is None:

        print("X Cannot print: Missing pulse data")

        if lcd:

            display_message(lcd, "Print Blocked", "No pulse data", 2)

        return False

    if hr < 40 or hr > 200:

        print(f"X Cannot print: Invalid HR ({hr})")

        if lcd:

            display_message(lcd, "Invalid HR", "Cannot print", 2)

        return False

    if spo2 < 70:

        print(f"X Cannot print: Invalid SpO2 ({spo2})")

        if lcd:
```

```

        display_message(lcd, "Invalid SpO2", "Cannot print", 2)

    return False

current_time = time.time()

if current_time - last_print_time < PRINT_COOLDOWN:

    remaining = int(PRINT_COOLDOWN - (current_time - last_print_time))

    print(f" ⌚ Print cooldown: {remaining}s remaining")

    if lcd:

        display_message(lcd, "Please wait...", f"{remaining}s cooldown", 1)

    return False

if (last_printed_temp is not None and

    abs(temp - last_printed_temp) < 0.3 and

    abs(hr - last_printed_hr) < 5 and

    abs(spo2 - last_printed_spo2) < 2):

    print("X Skipped: No significant change in readings")

    if lcd:

        display_message(lcd, "No Change", "Skipped print", 2)

    return False

steps = [

    ("Printing...", "Init printer"),

    ("Printing...", f"Patient #{patient_no}..."),

    ("Printing...", "Timestamp..."),

    ("Printing...", "Temp data..."),

    ("Printing...", "Heart rate..."),

    ("Printing...", "SpO2 data..."),

    ("Printing...", "Footer..."),

    ("Done!", "Check paper")

```

```
]
```

```
def show_progress(step_idx):
```

```
    if lcd and step_idx < len(steps):
```

```
        line1, line2 = steps[step_idx]
```

```
        lcd.clear()
```

```
        lcd.cursor_pos = (0, 0)
```

```
        lcd.write_string(line1[:16])
```

```
        lcd.cursor_pos = (1, 0)
```

```
        lcd.write_string(line2[:16])
```

```
        time.sleep(0.3)
```

```
try:
```

```
    print(f"\n[PRINTING REPORT - Patient #{patient_no}]\n")
```

```
    print(f" Philippine Time: {format_ph_time()}")
```

```
    show_progress(0)
```

```
    printer.write(b'\x1b@')
```

```
    printer.write(b'\x1b2')
```

```
    printer.write(b'\x1b!\x00')
```

```
    printer.flush()
```

```
    time.sleep(0.3)
```

```
    show_progress(1)
```

```
    header = b"\n\n===== \n HEALTH REPORT \n===== \n"
```

```
    printer.write(header)
```

```
    if patient_no:
```

```
        patient_line = f"Patient No: {patient_no}\n"
```

```
        printer.write(patient_line.encode())
```

```
    printer.flush()
```

```
time.sleep(0.2)
show_progress(2)
timestamp = format_ph_time()
printer.write(f"Time: {timestamp} PH\n".encode())
printer.write(b"-----\n")
printer.flush()
time.sleep(0.2)
show_progress(3)
temp_line = f"Temp: {temp:.1f}C ({status})\n"
printer.write(temp_line.encode())
printer.flush()
time.sleep(0.2)
if hr is not None and hr > 0:
    show_progress(4)
    hr_line = f"Heart Rate: {int(hr)} bpm\n"
    printer.write(hr_line.encode())
    printer.flush()
    time.sleep(0.2)
    show_progress(5)
    spo2_line = f"SpO2: {int(spo2)}%\n"
    printer.write(spo2_line.encode())
    printer.flush()
    time.sleep(0.2)
else:
    show_progress(4)
    printer.write(b"Heart Rate: N/A\n")
```

```

    printer.flush()
    time.sleep(0.2)
    show_progress(5)
    printer.write(b"SpO2: N/A\n")
    printer.flush()
    time.sleep(0.2)
    show_progress(6)
    footer = b"=====\n  Get Well Soon! \n=====\n"
    printer.write(footer)
    printer.flush()
    time.sleep(0.2)
    printer.write(b'\n\n\n\n')
    printer.flush()
    time.sleep(0.3)
    show_progress(7)
    last_print_time = current_time
    last_printed_temp = temp
    last_printed_hr = hr
    last_printed_spo2 = spo2
    print("✓ Print complete")
    return True
except serial.SerialTimeoutException:
    print("X Print timeout")
    if lcd:
        display_message(lcd, "Print Timeout", "Check paper", 2)
    return False

```

```
except Exception as e:

    print(f"X Print error: {e}")

    if lcd:

        display_message(lcd, "Print Error", str(e)[:14], 2)

    return False
```

```
# =====
```

```
# I2C SETUP
```

```
# =====
```

```
try:
```

```
    smbus = SMBus(1)

    mlx = MLX90614(smbus, address=0x5A)

    print("✓ MLX90614 temperature sensor initialized")
```

```
except Exception as e:
```

```
    print(f"X MLX90614 Error: {e}")

    mlx = None
```

```
try:
```

```
    i2c = busio.I2C(board.SCL, board.SDA)

    print("✓ I2C bus initialized")
```

```
except Exception as e:
```

```
    print(f"X I2C Error: {e}")

    i2c = None
```

```
# =====
```

```
# LCD DISPLAY
```

```
# =====

def init_lcd():

    addresses = [0x27, 0x3F, 0x20, 0x38]

    for addr in addresses:

        try:

            lcd = CharLCD(

                i2c_expander='PCF8574', address=addr, port=1,

                cols=16, rows=2, charmap='A00',

                auto_linebreaks=True, backlight_enabled=True

            )

            lcd.clear()

            print(f"✓ LCD display initialized at 0x{addr:02X}")

            return lcd

        except:

            continue

    print("✗ LCD Error: No device found")

    return None
```

```
lcd = init_lcd()
```

```
# =====

# MAX30102 PULSE OXIMETER

# =====

try:

    if MAX30102:

        sensor = MAX30102()
```

```

        print("✓ MAX30102 pulse oximeter initialized")
    else:
        sensor = None
except Exception as e:
    print(f"X MAX30102 Error: {e}")
    sensor = None

# =====
# PRINTER INITIALIZATION
# =====

printer = initialize_printer()

# =====
# BUZZER SETUP
# =====

try:
    GPIO.setmode(GPIO.BCM)
    GPIO.setwarnings(False)
    GPIO.setup(BUZZER_PIN, GPIO.OUT)
    buzzer_pwm = GPIO.PWM(BUZZER_PIN, 1000)
    buzzer_pwm.start(0)
    print("✓ Buzzer initialized")
except Exception as e:
    print(f"X Buzzer Error: {e}")
    buzzer_pwm = None

```



```
def beep(duration=0.2, repeat=1, frequency=1000):
```

```
    if not buzzer_pwm:
```

```
        return
```

```
    for _ in range(repeat):
```

```
        buzzer_pwm.ChangeFrequency(frequency)
```

```
        buzzer_pwm.ChangeDutyCycle(50)
```

```
        time.sleep(duration)
```

```
        buzzer_pwm.ChangeDutyCycle(0)
```

```
        time.sleep(duration / 2)
```

```
def beep_trigger():
```

```
    """3 short beeps when finger triggers the system"""
```

```
    beep(0.05, 3, 2000)
```

```
def beep_valid_reading():
```

```
    """Happy tone for valid readings (temp LOW/NORMAL/FEVER/HIGH, HR 60-130, SpO2  
>=95%)"""
```

```
    beep(0.1, 1, 1500)
```

```
    time.sleep(0.1)
```

```
    beep(0.1, 1, 2000)
```

```
    time.sleep(0.1)
```

```
    beep(0.15, 1, 2500)
```

```
def beep_invalid_reading():
```

```
    """No beep for invalid readings - silent"""
```

```
    pass # Silent for invalid readings
```

```
def beep_fever():
```

```
    beep(0.8, 1, 800)
```

```
def beep_high_fever():
```

```
    beep(0.15, 5, 1200)
```

```
def beep_success():
```

```
    beep(0.1, 2, 2000)
```

```
def beep_body_detected():
```

```
    beep(0.1, 1, 1500)
```

```
def beep_startup():
```

```
    beep(0.1, 1, 1000)
```

```
    time.sleep(0.1)
```

```
    beep(0.1, 1, 1500)
```

```
    time.sleep(0.1)
```

```
    beep(0.1, 1, 2000)
```

```
def beep_print_success():
```

```
    beep(0.1, 2, 2000)
```

```
def beep_cloud_success():
```

```
    beep(0.05, 3, 2500)
```

```

def beep_cloud_failed():
    beep(0.3, 3, 600)

# =====

# VALIDATION FUNCTIONS

# =====

def is_valid_temperature(temp):
    """Check if temperature is in valid range (36.0 - 37.4°C)"""
    return VALID_TEMP_MIN <= temp <= VALID_TEMP_MAX

def is_temperature_readable(temp, status):
    """Check if temperature reading is valid for buzzer (any readable status: LOW, NORMAL,
    FEVER, HIGH)"""
    return status in VALID_TEMP_STATUSES and temp is not None

def is_valid_hr(hr):
    """Check if heart rate is in valid range (60-180 bpm)"""
    return VALID_HR_MIN <= hr <= VALID_HR_MAX

def is_valid_spo2(spo2):
    """Check if SpO2 is in valid range (>=95%)"""
    return spo2 >= VALID_SPO2_MIN

def check_all_valid(temp, hr, spo2, temp_status=None):
    """Check if all vitals are valid - returns tuple of (all_valid, details)"""
    # For temperature, accept any readable status (LOW, NORMAL, FEVER, HIGH)

```

```
temp_valid = is_temperature_readable(temp, temp_status) if temp_status else  
is_valid_temperature(temp)
```

```
hr_valid = is_valid_hr(hr)
```

```
spo2_valid = is_valid_spo2(spo2)
```

```
details = {  
    'temp': temp_valid,  
    'hr': hr_valid,  
    'spo2': spo2_valid  
}
```

```
all_valid = temp_valid and hr_valid and spo2_valid
```

```
return all_valid, details
```

```
# =====
```

```
# TEMPERATURE READING (NO AMBIENT)
```

```
# =====
```

```
def get_raw_temperature():
```

```
    """Get raw object temperature only (no ambient reading)"""
```

```
    if not mlx:
```

```
        return None
```

```
    try:
```

```
        # Only read object temperature, skip ambient
```

```
        object_temp = mlx.get_obj_temp()
```

```
        return object_temp
```

```
    except Exception as e:
```

```
print(f"Temperature read error: {e}")
```

```
return None
```

```
def is_body_detected_with_restart_check(timeout=BODY_DETECTION_TIMEOUT,  
threshold=BODY_DETECTION_THRESHOLD, samples=BODY_DETECTION_SAMPLES):
```

```
    global restart_requested
```

```
    if not mlx:
```

```
        return False
```

```
    print(" Waiting for body detection... (press RESTART to reset sensors)")
```

```
    start_time = time.time()
```

```
    detection_count = 0
```

```
    while time.time() - start_time < timeout:
```

```
        if check_restart_button():
```

```
            restart_requested = True
```

```
            print(" ↳ Restart detected during body detection wait")
```

```
            return False
```

```
    try:
```

```
        object_temp = get_raw_temperature()
```

```
        if object_temp is None:
```

```
            time.sleep(0.1)
```

```
            continue
```

```
        if object_temp > threshold:
```

```
            detection_count += 1
```

```
            print(f" Object temp: {object_temp:.1f}°C (detect {detection_count}/{samples})")
```

```
            if detection_count >= samples:
```

```
                print(f" ✓ Body detected at {get_ph_time_short()} PH! Object:  
{object_temp:.1f}°C")
```

```

        return True

    else:

        if detection_count > 0:

            print(f"  Lost detection ({object_temp:.1f}°C < {threshold}°C)")

            detection_count = 0

            time.sleep(0.2)

        except Exception as e:

            print(f"  Detection error: {e}")

            time.sleep(0.2)

    print(f" X Body detection timeout at {get_ph_time_short()} PH")

    return False

```

```

def get_body_temperature():

    """Get body temperature using raw object reading (no ambient compensation)"""

    if not mlx:

        return None, "NO SENSOR"

    try:

        samples = []

        for _ in range(10):

            object_temp = get_raw_temperature()

            if object_temp is not None:

                # Use raw object temperature directly - NO ambient compensation

                samples.append(object_temp)

            time.sleep(0.05)

        if not samples:

            return None, "ERROR"

```

```

samples.sort()

temperature = samples[len(samples)//2]

if temperature < 30 or temperature > 45:

    return None, "ERROR"

    # CALIBRATION: Add +2°C to any reading to normalize

temperature = temperature + 2.0

if temperature <= 37.4:

    status = "NORMAL"

elif temperature <= 38.4:

    status = "FEVER"

else:

    status = "HIGH"

return temperature, status

except Exception as e:

    print(f"Temperature read error: {e}")

    return None, "ERROR"

```

```

# =====

```

```

# FINGER TRIGGER SYSTEM

```

```

# =====

```

```

def display_standby_screen(lcd):

    """Show standby screen waiting for finger trigger"""

    if not lcd:

        return

    lcd.clear()

    lcd.cursor_pos = (0, 0)

```

```
lcd.write_string("INSERT FINGER")
```

```
lcd.cursor_pos = (1, 0)
```

```
lcd.write_string("TO START  ")
```

```
def wait_for_finger_trigger():
```

```
    """
```

```
    Wait indefinitely for finger on MAX30102 to trigger measurement.
```

```
    Returns True when finger detected, False if shutdown/restart requested.
```

```
    """
```

```
    global restart_requested, shutdown_requested
```

```
    if not sensor:
```

```
        print("X MAX30102 not available - cannot use finger trigger")
```

```
        return True # Continue without trigger if no sensor
```

```
    print("\n 🖐 FINGER TRIGGER MODE: Waiting for finger...")
```

```
    print(f" Threshold: IR > {FINGER_TRIGGER_THRESHOLD}")
```

```
    print(f" Required consecutive readings: {FINGER_TRIGGER_COUNT}")
```

```
    detection_count = 0
```

```
    last_ir_value = 0
```

```
    while True:
```

```
        # Check power switch
```

```
        if check_power_switch():
```

```
            return False
```



```

# Check restart button

if check_restart_button():

    restart_requested = True

    return False


# Process restart if requested

if restart_requested:

    restart_requested = False

    force_restart_sensors()

    display_standby_screen(lcd)


try:

    # Read from MAX30102

    data = sensor.read_fifo()

    if data and len(data) >= 2:

        ir_value = data[0]

        last_ir_value = ir_value


    # Check if finger detected (IR above threshold)

    if ir_value > FINGER_TRIGGER_THRESHOLD:

        detection_count += 1


    # Update LCD with detection progress

    if lcd and detection_count > 0:

        lcd.clear()

```

```

        lcd.cursor_pos = (0, 0)

        lcd.write_string(f"IR: {ir_value:>6}")

        lcd.cursor_pos = (1, 0)

        lcd.write_string(f"Detect {detection_count}/{FINGER_TRIGGER_COUNT}")


    print(f" Finger detected: IR={ir_value}
    ({detection_count}/{FINGER_TRIGGER_COUNT})")


    # Require consecutive readings to confirm
    if detection_count >= FINGER_TRIGGER_COUNT:

        print(f"✓ FINGER TRIGGER CONFIRMED at {get_ph_time_short()} PH!")

        beep_trigger()

        if lcd:

            display_message(lcd, "Finger OK!", "Starting...", 1)

            return True

    else:

        # Reset counter if reading drops below threshold
        if detection_count > 0:

            print(f" Lost finger: IR={ir_value} (< {FINGER_TRIGGER_THRESHOLD})")

            detection_count = 0


    # Show standby screen

    if lcd:

        display_standby_screen(lcd)


time.sleep(FINGER_TRIGGER_CHECK_INTERVAL)

```

```

except Exception as e:

    print(f" Trigger read error: {e}")

    time.sleep(FINGER_TRIGGER_CHECK_INTERVAL)


# =====

# FINGER DETECTION (for pulse reading)

# =====

def is_finger_detected_with_restart_check(threshold=50000, timeout=2.0):

    global restart_requested

    if not sensor:

        return False

    start_time = time.time()

    detection_count = 0

    while time.time() - start_time < timeout:

        if check_restart_button():

            restart_requested = True

            return False

    try:

        data = sensor.read_fifo()

        if data and len(data) >= 2:

            ir = data[0]

            if ir > threshold:

                detection_count += 1

                if detection_count >= 3:

                    return True

```

```

        else:

            detection_count = 0

    except:

        pass

    time.sleep(0.05)

return False


# =====

# PULSE DATA COLLECTION - ORIGINAL (UNCHANGED)

# =====

def collect_pulse_data(samples=100, interval=0.03):

    """

    ORIGINAL FUNCTION - UNCHANGED

    Collect pulse data from MAX30102

    """

    if not sensor:

        return None, None

    red_data = []

    ir_data = []

    for _ in range(samples):

        try:

            data = sensor.read_fifo()

            if data and len(data) >= 2:

                ir_data.append(data[0])

                red_data.append(data[1])

        else:

```

```

        ir_data.append(0)
        red_data.append(0)
    except:
        ir_data.append(0)
        red_data.append(0)
    time.sleep(interval)
    return ir_data, red_data

# =====

# DISPLAY FUNCTIONS

# =====

def display_temperature(lcd, temp, status, patient_no=None):
    if not lcd:
        return
    lcd.cursor_pos = (0, 0)
    if patient_no:
        lcd.write_string(f"#{patient_no} T:{temp:.1f}C ")
    else:
        lcd.write_string(f"Temp:{temp:.1f}C ")
    lcd.cursor_pos = (1, 0)
    lcd.write_string(f"{status: <16}")

def display_pulse(lcd, hr, spo2, patient_no=None):
    if not lcd:
        return
    lcd.clear()

```

```

    lcd.cursor_pos = (0, 0)

    if patient_no:
        lcd.write_string(f"#{patient_no} HR:{int(hr):3d} ")
    else:
        lcd.write_string(f"HR:{int(hr):3d} bpm ")

    lcd.cursor_pos = (1, 0)

    lcd.write_string(f"SpO2:{int(spo2):2d}% ")

def display_message(lcd, line1, line2="", duration=2):
    if not lcd:
        print(f"LCD: {line1} | {line2}")
        time.sleep(duration)
        return

    lcd.clear()

    lcd.cursor_pos = (0, 0)

    lcd.write_string(line1[:16])

    if line2:
        lcd.cursor_pos = (1, 0)

        lcd.write_string(line2[:16])

    if duration > 0:
        time.sleep(duration)

# =====

# MAIN LOOP

# =====

def main():

```

```
global current_patient_no, restart_requested, shutdown_requested, startup_complete,  
power_switch_enabled
```

```
print("\n" + "="*50)  
print("RASPBERRY PI 4 HEALTH MONITOR - PHILIPPINE TIME")  
print("="*50)  
print(f"Start Time: {format_ph_time()} (Philippine Time)")
```

```
# STEP 1: Setup GPIO pins
```

```
if not setup_gpio():  
    print("WARNING: GPIO setup failed...")  
    return
```

```
# STEP 2: Check switch position
```

```
power_state = GPIO.input(POWER_SWITCH_PIN)  
  
print(f"\n 🚦 Power switch state: {'ON (HIGH)' if power_state == GPIO.HIGH else 'OFF  
(LOW)'}")
```

```
if power_state == GPIO.LOW:
```

```
    print("\n ⚠️ WARNING: Power switch is in OFF position!")  
    print(" Please turn the switch ON to start the system.")  
    print(" Waiting for switch to be turned ON...")  
    while GPIO.input(POWER_SWITCH_PIN) == GPIO.LOW:  
        if lcd:  
            lcd.clear()  
            lcd.cursor_pos = (0, 0)
```

```
lcd.write_string("Turn Power ON")

lcd.cursor_pos = (1, 0)

lcd.write_string("to start system ")

time.sleep(0.5)

print("\n✓ Power switch turned ON - continuing startup")

time.sleep(1)
```

STEP 3: Short delay to stabilize

```
print("\n ⌚ System stabilizing... (3 seconds)")
```

```
for i in range(3, 0, -1):
```

```
    print(f" Starting in {i} seconds...", end="\r")
```

```
    time.sleep(1)
```

```
print("\n✓ Stabilization complete      ")
```

STEP 4: Mark startup complete and enable power switch

```
startup_complete = True
```

```
power_switch_enabled = True
```

```
print("\n✓ System fully started")
```

```
print("\nControls:")
```

```
print(f" • FINGER TRIGGER: Place finger on MAX30102 to start measurement")
```

```
print(f" • RESTART Button: GPIO {RESTART_BUTTON_PIN} (Pin 13) - Press to reset  
sensors")
```

```
print(f" • POWER Switch: GPIO {POWER_SWITCH_PIN} (Pin 11) - Turn OFF and hold 5s to  
shutdown")
```

```
print(f"\nValid Reading Criteria (Buzzer will sound):")
```

```
print(f" • Temperature: LOW, NORMAL, FEVER, HIGH (all readable temps, NO ambient  
comp)")
```



```
print(f" • Heart Rate: {VALID_HR_MIN} - {VALID_HR_MAX} bpm")
print(f" • SpO2: >= {VALID_SPO2_MIN}%")

# Startup beep and display
beep_startup()
display_message(lcd, "Health Monitor", f"{get_ph_time_short()} PH Ready", 2)

# Show printer status
if printer:
    print("\n✓ Printer: READY")
else:
    print("\nX Printer: NOT AVAILABLE")

# Show network status
try:
    test_response = requests.get("https://google.com", timeout=5)
    print("✓ Network: CONNECTED")
except:
    print("X Network: NOT AVAILABLE")

cycle_count = 0

try:
    while True:
        cycle_count += 1
```

```
# === FINGER TRIGGER MODE ===

# Wait for finger trigger to start measurement cycle
print(f"\n--- Cycle {cycle_count} | Waiting for finger trigger ---")

# Show standby screen
display_standby_screen(lcd)

# Wait indefinitely for finger trigger
trigger_result = wait_for_finger_trigger()

if shutdown_requested:
    safe_shutdown()
    return

if restart_requested:
    restart_requested = False
    force_restart_sensors()
    continue

if not trigger_result:
    # Trigger was cancelled or error occurred
    continue

# === MEASUREMENT CYCLE STARTS HERE ===

# Get current patient number
patient_no = get_current_patient_no()
```

```
current_time_str = get_ph_time_short()

print(f"\n>>> MEASUREMENT START | Patient #{patient_no} | {current_time_str} PH
<<<")
```

```
# Display patient menu
```

```
display_patient_menu(lcd)
```

```
time.sleep(1)
```

```
# STEP 1: WAIT FOR BODY DETECTION
```

```
display_message(lcd, f"Patient #{patient_no}", "Place body part", 0)
```

```
if not is_body_detected_with_restart_check():
```

```
    if restart_requested:
```

```
        restart_requested = False
```

```
        force_restart_sensors()
```

```
        display_message(lcd, "Sensors Reset", f"Patient #{current_patient_no}", 2)
```

```
        continue
```

```
    display_message(lcd, "No body detected", "Try again...", 2)
```

```
    continue
```

```
# Check restart button again
```

```
if check_restart_button():
```

```
    restart_requested = True
```

```
    restart_requested = False
```

```
    force_restart_sensors()
```

```
    display_message(lcd, "Sensors Reset", f"Patient #{current_patient_no}", 2)
```

```
continue
```

```
beep_body_detected()
```

```
display_message(lcd, "Body detected!", "Reading temp...", 1)
```

```
# STEP 2: READ TEMPERATURE
```

```
temperature, status = get_body_temperature()
```

```
if temperature is None:
```

```
    display_message(lcd, "TEMP ERROR", "Retry...", 2)
```

```
    continue
```

```
    print(f"Patient #{patient_no} | Temp: {temperature:.1f}°C ({status}) |  
{get_ph_time_short()} PH")
```

```
    display_temperature(lcd, temperature, status, patient_no)
```

```
# Temperature is always considered valid if it has a readable status
```

```
temp_readable = is_temperature_readable(temperature, status)
```

```
if not temp_readable:
```

```
    print(f" ⚠ Temperature reading error - No beep")
```

```
if status == "FEVER":
```

```
    beep_fever()
```

```
elif status == "HIGH":
```

```
    beep_high_fever()
```

```
if temperature < 30 or temperature > 45:
```

```
    display_message(lcd, "Invalid Temp", "Retry...", 2)
```

```
    time.sleep(2)
```

```
    continue
```

```
time.sleep(2)
```

```
# STEP 3: WAIT FOR FINGER (for pulse)
```

```
hr, spo2 = 0, 0
```

```
pulse_success = False
```

```
pulse_valid = False
```

```
if sensor:
```

```
    display_message(lcd, f"Patient #{patient_no}", "Place finger", 0)
```

```
if is_finger_detected_with_restart_check():
```

```
    if restart_requested:
```

```
        restart_requested = False
```

```
        force_restart_sensors()
```

```
        display_message(lcd, "Sensors Reset", f"Patient #{current_patient_no}", 2)
```

```
        continue
```

```
beep_success()
```

```
display_message(lcd, "Reading...", "Keep still", 1)
```

```
# === ORIGINAL PULSE DATA COLLECTION - UNCHANGED ===
```

```

ir_data, red_data = collect_pulse_data()

if ir_data and hrcalc:

    hr_calc, hr_valid, spo2_calc, spo2_valid = hrcalc.calc_hr_and_spo2(ir_data,
red_data)

    if hr_valid and 30 <= hr_calc <= 250 and spo2_valid and spo2_calc >= 70:

        hr = hr_calc

        spo2 = spo2_calc

        pulse_success = True

    # Check if pulse readings are valid for buzzer

    hr_valid_range = is_valid_hr(hr)

    spo2_valid_range = is_valid_spo2(spo2)

    pulse_valid = hr_valid_range and spo2_valid_range

    print(f"Patient #{patient_no} | HR: {int(hr)} bpm | SpO2: {int(spo2)}% |
{get_ph_time_short()} PH")

    if not hr_valid_range:

        print(f" ⚠️ HR {int(hr)} outside valid range ({VALID_HR_MIN}-
{VALID_HR_MAX}) - No beep")

    if not spo2_valid_range:

        print(f" ⚠️ SpO2 {int(spo2)}% below valid threshold ({VALID_SPO2_MIN}%
- No beep")

    display_pulse(lcd, hr, spo2, patient_no)

```

```

        beep(0.05, 3, 1500)

        time.sleep(4)

    else:

        print(f"Patient #{patient_no} | Invalid pulse data | {get_ph_time_short()} PH")

        display_message(lcd, "Invalid Pulse", "Continue...", 2)

    else:

        display_message(lcd, "Pulse Error", "Continue...", 2)

else:

    if restart_requested:

        restart_requested = False

        force_restart_sensors()

        display_message(lcd, "Sensors Reset", f"Patient #{current_patient_no}", 2)

        continue

    display_message(lcd, "No finger", "Temp only...", 2)

else:

    display_message(lcd, "No pulse sensor", "Temp only mode", 2)


# === CHECK ALL VALIDATIONS AND BEEP IF VALID ===

all_valid, details = check_all_valid(temperature, hr, spo2, status)

if all_valid and pulse_success:

    print(f"\n  ALL READINGS VALID - Buzzer ON")

    print(f" Temp: {temperature:.1f}°C ({status}) ✓ | HR: {int(hr)} bpm ✓ | SpO2: {int(spo2)}% ✓")

    beep_valid_reading()

# MODIFIED: Show "Valid Reading" for all valid temperature statuses

```

```

if lcd:

    display_message(lcd, "Valid Reading!", f"T:{temperature:.1f} HR:{int(hr)}", 2)

elif not pulse_success:

    # Temperature only mode - check temp only (accept any readable status: LOW,
    NORMAL, FEVER, HIGH)

    if is_temperature_readable(temperature, status):

        print(f"\n✅ TEMPERATURE VALID ({status}) - Buzzer ON")

        print(f" Temp: {temperature:.1f}°C ✓")

        beep_valid_reading()

        # MODIFIED: Show "Valid Reading" for all valid temperature statuses

        if lcd:

            display_message(lcd, "Valid Reading!", f"{temperature:.1f}°C", 2)

        else:

            print(f"\n❌ TEMPERATURE INVALID - Buzzer OFF")

            beep_invalid_reading()

    else:

        print(f"\n❌ READINGS INVALID - Buzzer OFF (Silent)")

        print(f" Temp: {temperature:.1f}°C {'✓' if details['temp'] else 'X'} | "

              f"HR: {int(hr)} bpm {'✓' if details['hr'] else 'X'} | "

              f"SpO2: {int(spo2)}% {'✓' if details['spo2'] else 'X'}")

        beep_invalid_reading()

        if lcd:

            display_message(lcd, "Invalid Reading", "Check values", 2)

# Check restart button before printing

if check_restart_button():

```



```
restart_requested = True
```

```
restart_requested = False
```

```
force_restart_sensors()
```

```
display_message(lcd, "Sensors Reset", f"Patient #{current_patient_no}", 2)
```

```
continue
```

```
# STEP 4: PRINT REPORT & SEND TO CLOUD
```

```
print_success = False
```

```
cloud_sent = False
```

```
if printer:
```

```
    print_success = print_with_progress(printer, temperature, status, hr, spo2, lcd,  
patient_no)
```

```
if print_success:
```

```
    beep_print_success()
```

```
    display_message(lcd, "Syncing...", "Cloud upload", 1)
```

```
    cloud_sent = send_vitals_to_supabase(  
        patient_no=patient_no,  
        temperature=temperature,  
        temp_status=status,  
        heart_rate=hr if hr > 0 else None,  
        spo2=spo2 if spo2 > 0 else None  
    )
```

```
if cloud_sent:
```

```

        beep_cloud_success()

        display_message(lcd, "Cloud Sync", "Success!", 1)
    else:

        beep_cloud_failed()

        display_message(lcd, "Cloud Sync", "Failed", 1)

    increment_patient_no()
else:

    beep_cloud_failed()

    print(f"Print failed - keeping Patient #{patient_no}")
else:

    display_message(lcd, "No Printer", "Skip print", 2)

    print(f"No printer - keeping Patient #{patient_no}")

# === RETURN TO STANDBY ===

print(f"\n✓ MEASUREMENT COMPLETE - RETURNING TO STANDBY")

display_message(lcd, "Done!", "Remove finger", 2)

time.sleep(1)

if lcd:

    lcd.clear()

except KeyboardInterrupt:

    print(f"\nShutdown at {format_ph_time()} PH Time...")
finally:

    cleanup()

```

```
def cleanup():
    print(f"Cleaning up at {format_ph_time()} PH Time...")
    cleanup_gpio()
    if lcd:
        try:
            lcd.clear()
            lcd.write_string("Goodbye... ")
            time.sleep(1)
            lcd.clear()
            lcd.backlight_enabled = False
        except:
            pass
    if buzzer_pwm:
        buzzer_pwm.stop()
        GPIO.cleanup()
    if printer and printer.is_open:
        try:
            printer.write(b'\n\n\n\n\n')
            printer.flush()
            time.sleep(0.5)
            printer.close()
        except:
            pass
    print("Shutdown complete.")
```

```
if __name__ == "__main__":  
    main()
```